Quick Reference: Diagnosis-Driven Runbook

Actively tuning a load? Start with the Quick ADF Copy Activity Performance Tuning Runbook (Word) | Quick ADF Copy Activity Performance Tuning Runbook (PDF). Use this reference when a metric or bottleneck needs deeper interpretation before choosing the next test. For the complete methodology and remediation paths, use the Full ADF Copy Activity Performance Tuning Runbook (Word) | Full ADF Copy Activity Performance Tuning Runbook (PDF).

Use one rule throughout: change one variable, run three comparable executions, compare medians, and stop when the next change no longer produces useful improvement.

1. Measure only what drives a decision

Measure	What it tells you
Copy duration and throughput	Whether the change actually made the load faster
Rows copied / rejected	Whether the faster run is still correct
DIUs used and parallel copies	What ADF actually allocated/used
WRITELOG	Whether SQL MI sessions are waiting for transaction-log flushes
INSTANCE_LOG_RATE_GOVERNOR	Whether SQL MI log-write capacity is limiting further write throughput
Log write latency and live IOPS	Whether SQL MI storage/log I/O pressure increased
Cost per GiB	Whether the performance gain justifies the additional ADF resources

Decision thresholds

- PASS: median performance improves 10% or more, data is correct, and SQL MI does not develop a new limiting wait/latency condition.
- STOP: the next setting improves performance by less than 5%, or SQL MI waits/latency/cost increase substantially without a proportional throughput gain.
- CONFIRM: improvement is 5-10%. Repeat the comparison before accepting the setting.
- INVALID SERIES: the repeated baseline at the end differs by more than 15% from the starting baseline. The environment changed enough that the tuning comparison is unreliable.

Use ADF_Performance_Toolkit monitoring scripts 01-08 when needed. Do not run every diagnostic continuously. Use the focused SQL MI scripts when the ADF results indicate the sink may be limiting performance.

2. DIU tuning

Keep file count, batch size, target table, and parallel-copy setting unchanged.

Step	DIU setting	Runs	Decision
D0	Auto	3	Establish baseline median and record DIUs actually used.
D1	4	3	Continue if median performance improves >=10% or cost/GiB improves materially.
D2	8	3	Continue if another >=10% gain occurs and SQL MI waits/latency remain comparable.
D3	Next supported value	3	Stop at the first <5% incremental gain or when SQL MI pressure rises without proportional throughput.
D4	Auto	3	If this median differs >15% from D0, treat the series as unreliable and repeat it.

What the results mean

- Throughput increases >=10% and SQL MI waits/latency remain comparable: ADF data-movement compute was limiting or contributing. Test the next DIU level.
- Throughput increases <5% after raising DIUs: additional ADF compute is not producing useful scaling. Stop increasing DIUs and investigate source, network, or SQL MI sink limits.
- Throughput plateaus while WRITELOG or log write latency rises: SQL MI is accepting additional write pressure without producing proportional throughput. Stop increasing DIUs and investigate transaction-log performance.
- INSTANCE_LOG_RATE_GOVERNOR increases sharply while throughput improves <10%: SQL MI is spending increasing time waiting on its transaction-log write-rate limit. Additional DIUs or writers are mostly adding pressure instead of useful throughput. Stop increasing DIUs or write concurrency.

Example:

text

8 DIUs: 100 MiB/s, 1 second INSTANCE_LOG_RATE_GOVERNOR

16 DIUs: 104 MiB/s, 75 seconds INSTANCE_LOG_RATE_GOVERNOR

The 8 -> 16 DIU change gained only 4% throughput while governor waits increased sharply. Keep 8 DIUs as the candidate and investigate SQL MI log throughput before adding more ADF pressure.

- Higher DIUs cost more while throughput improves <5%: use the lower setting. The extra compute is not producing enough useful work to justify the additional cost.

Production DIU acceptance

1. Capture at least three untouched production executions as P0.
2. Apply one lab-qualified DIU change only.
3. Validate row counts and rejected rows before evaluating performance.
4. Compare median duration, throughput, SQL MI waits/latency, and cost against P0.
5. Promote only when the result improves >=10% with correct data and no unacceptable SQL MI or cost regression. Otherwise rollback or investigate.

3. Write batch sizing

Batch sizing tests commit frequency and transaction-log pressure under SQL MI FULL recovery.

Step	Batch size	Runs	Decision
B0	Default/unset	3	Record baseline duration, throughput, WRITELOG, active log usage, and log latency.
B1	10,000	3	Keep if >=10% useful improvement occurs without disproportionate log growth.
B2	50,000	3	Continue only if performance still scales and log pressure remains controlled.
B3	100,000	3	Stop if gain <5%, WRITELOG/latency rises sharply, active log grows disproportionately, or rollback becomes operationally unacceptable.
B4	Higher only if B3 still scales	3	A larger batch must demonstrate another useful gain; do not assume 250,000 is better.
B5	Default/unset	3	>15% baseline drift invalidates the comparison series.

What the results mean

- Larger batch improves throughput >=10% while log usage and latency remain comparable: commit overhead was contributing to the load time. Continue one controlled step.
- Larger batch improves throughput <5% while active log usage increases: the larger transaction is consuming more log without delivering useful speed. Return to the smaller batch.
- ACTIVE_TRANSACTION persists and active log space grows after increasing batch size: the larger transaction is retaining log space longer. Reduce batch size unless the performance benefit clearly justifies that operational risk.
- A failed load takes substantially longer to roll back after increasing batch size: the larger transaction increased recovery exposure. Treat that as a production cost even if the successful load is faster.

Production batch acceptance

6. Record the current batch size and three representative P0 runs.
7. Choose one neighboring lab-qualified batch size. Do not skip multiple levels.
8. Change batch size only.
9. Validate exact row counts, rejected rows, and target integrity.
10. Compare duration/throughput plus WRITELOG, active log usage, log latency, rollback behavior, and cost.
11. PASS only for >=10% useful median improvement with acceptable operational behavior. FAIL rolls back. INVESTIGATE is not promoted.

4. Parallelism and file counts

Test these separately. DIUs, parallel copies, source-file count, and simultaneous SQL writers are different variables.

Parallel copies

Keep DIUs, batch size, and source layout fixed.

Step	Parallel copies	Runs	Decision
P0	Auto	3	Baseline ADF-selected behavior.
P1	1	3	Establish the serialized reference.
P2	2	3	Continue if >=10% end-to-end gain occurs and SQL MI remains comparable.
P3	4	3	Stop if incremental gain <5% or lock/log pressure rises without proportional throughput.
P4	Next supported value only if P3 scales	3	Stop at the first useful-performance knee.
P5	Auto	3	>15% drift invalidates the series.

What the parallelism results mean

- 2 copies improve end-to-end throughput >=10% over 1 with similar SQL MI waits: concurrency is useful. Test 4.
- 4 copies improve <5% over 2: the useful concurrency knee is approximately 2 for this configuration. Prefer 2 rather than adding unnecessary pressure.
- LCK_M_X or LCK_M_IX rises sharply while throughput plateaus: concurrent writers are contending for locks. Reduce write concurrency; adding DIUs is not the next fix.
- INSTANCE_LOG_RATE_GOVERNOR rises sharply while throughput improves <10%: concurrent writers are pushing SQL MI into its log-rate limit. Reduce concurrency or investigate SQL MI capacity/design before adding writers.
- ADF/source throughput rises but end-to-end load throughput does not: the source is no longer the limiting component. Stop adding source-side concurrency and investigate the downstream sink.

File counts

Use the same logical dataset. Create partitions with split_delimited_file.py and verify that partition row counts sum exactly to the original row count.

Step	Files	Runs	Decision
F0	1	3	Canonical source-layout baseline.
F1	2	3	Keep only if end-to-end performance improves.
F2	4	3	Continue if >=10% gain occurs with comparable SQL MI pressure.
F3	8	3	Test only if 4 files still scale or production naturally receives many files.
F4	16 only if 8 still scales	3	Do not equate 16 source files with 16 simultaneous SQL writers.
F5	1	3	>15% drift invalidates the series.

What the file-count results mean

- 4 files are >=10% faster than 1 and SQL MI pressure remains comparable: file-level/source parallelism is useful. Test 8 only if there is value in further scaling.
- 4 files perform within 5% of 1 file: partitioning is not addressing the current bottleneck. Prefer the simpler one-file design unless another operational requirement favors partitioning.
- More files increase source activity while end-to-end duration stops improving and SQL MI waits rise: source parallelism has saturated the sink. Stop increasing file count or write concurrency.
- 8 files are slower than 4: partition-management overhead or sink contention now exceeds the parallelism benefit. Use 4 for this tested configuration.

5. Final confirmation and production promotion

After tuning each variable independently, test the selected DIU + batch + parallelism + file-count configuration together three times. Then repeat the original untouched baseline three times.

Outcome	Rule
PASS	>=10% median improvement, exact data, no unacceptable new SQL MI limiting condition, and acceptable cost.
FAIL	Incorrect data, reliability regression, unacceptable sink pressure, or no useful performance improvement. Restore the previous accepted configuration.
INVESTIGATE	Performance improves, but cost or a displaced bottleneck needs explanation. Do not promote the configuration yet.

Production sequence

12. Capture P0 baseline.
13. State the specific hypothesis being tested.
14. Change one variable.
15. Validate correctness first.
16. Measure ADF and SQL MI.
17. Compare median performance and cost against P0.
18. Check whether the change simply moved the bottleneck downstream.
19. Classify PASS, FAIL, or INVESTIGATE.
20. Only a PASS becomes the new production baseline.

Production workload context: the referenced production feed is approximately 134.04 GiB and 78,949,101 rows. At that scale, sustained transaction-log throughput, restart/recovery behavior, reliability, and correctness matter more than short-run lab speed.

Writing standard for this runbook

State the measured condition, the obvious technical issue it indicates, and the action to take. Avoid vague phrases such as "becomes material," "looks high," or "appears constrained" unless the next sentence defines the condition with a measurable comparison.